Einführung in die Rechnerarchitektur

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Überblick der nächsten Wochen

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Überblick der nächsten Wochen

Überblick der nächsten Wochen

- Etablierte Sicht: Rechner ist eine „Kiste“, die mein Programm „wie gewünscht“ ausführt
- Tatsächlich: Rechner ist eine Schaltung, die „lediglich“ mit „Strom an“ und “Strom aus“ arbeitet

```
int a;
int b;
int c;
int d;

a = b * c;
b = (b + c) - d;

if ( a == b ) {
    cout << “Case 1”;
} else {

    cout << “Case 2”;
}
...
```

?

Überblick der nächsten Wochen

- Höhere Programmiersprache
- Assembly
- Maschinensprache

```
int a;
int b;
int c;
int d;

a = b * c;
b = (b + c) - d;

if ( a == b ) {
    cout << “Case 1”;
} else {

    cout << “Case 2”;
}
...
```

```
#a liegt im Speicherblock $t0
#b liegt im Speicherblock $t1
#c liegt im Speicherblock $t2
#d liegt im Speicherblock $t3

mul $t0 $t1 $t2
add $t1 $t1 $t2
sub $t1 $t1 $t3
bneq $t0 $t1 ELSE
syscall … # “Case 1”
j ENDIF
ELSE:
syscall # … “Case 2”
ENDIF
…
```

```
Daten
(im Datenspeicher in den
entsprechenden Blöcken)
Programm
(im Programmspeicher)
000100 00000 00001 00010 …
000000 00001 00001 00010 …
000010 00001 00001 00011 …
001000 00000 00001 1001001010…
100000 0101010101111011101000…
110000 00000 00001 1001001010…

100000 010101010111010 0…

…
```

Überblick der nächsten Wochen

- Wie kann ich Rechnen?
- Wie kann ich Speichern?
- Wie kann ich Steuern?
- Wie füge ich das in einem Prozessor zusammen?
- Wie entwerfe ich Schaltungen dafür?

→ Thema der kommenden Wochen

- Assembly

```
#a liegt im Speicherblock $t0
#b liegt im Speicherblock $t1
#c liegt im Speicherblock $t2
#d liegt im Speicherblock $t3

mul $t0 $t1 $t2
add $t1 $t1 $t2
sub $t1 $t1 $t3
bneq $t0 $t1 ELSE
syscall … # “Case 1”
j ENDIF
ELSE:
syscall # “Case 2”
ENDIF
…
```

- Maschinensprache

```
Daten
(im Datenspeicher in den
entsprechenden Blöcken)
Programm
(im Programmspeicher)
000100 00000 00001 00010 …
000000 00001 00001 00010 …
000010 00001 00001 00011
001000 00000 00001 1001001010…
100000 0101010101110111101000…
110000 00000 00001 1001001010…

100000 0101010101110100…
…
```

Überblick der nächsten Wochen

=

- **Grundlagen**
  - Beschreibungen über „0“ und „1“ (Boolesche Algebra)
  - Beschreibungen von Schaltungen
- **Rechnen**
  - Digitale Schaltungen für Addition, Subtraktion, Multiplikation
- **Speichern & Steuern**
  - Sequentielle Schaltungen
  - Speicherelemente
  - Endliche Automaten
- **Prozessor**
  - Aufbau
  - Optimierungen
- **Entwerfen**
  - Synthese von allgemeinen Schaltungen
  - Logikminimierung

Grundlagen: Boolesche Algebra

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Aussagen

- Aussagen haben einen Wahrheitswert
  - „10 ist eine gerade Zahl“ (wahr)
  - „9 ist eine Primzahl“ (falsch)
  - Aber: „Freds Schwester“ ist keine Aussage
- Aussagen können durch Platzhalter symbolisiert werden (Aussagevariablen)
  - a = „10 ist eine gerade Zahl“
  - b = „9 ist eine Primzahl“
- Bezeichnungen für Wahrheitswerte
  - Wahr, w, true, 1
  - Falsch, f, false, 0
- Aussagen können miteinander verknüpft werden
  → ***Aussagenlogik***

Logische Verknüpfungen #1

- <u>Einstelliger Operator:</u>
  Negation
  Umkehrung des Wahrheitswertes
  - Symbol: ¬ oder Querstrich über der Aussage
  - Beispiel: $\neg a$, $\overline{b}$

- <u>Zweistellige Operatoren:</u>
  Und, Oder, Äquivalenz, Antivalenz, Implikation

Logische Verknüpfungen #2

- **Und-Verknüpfung**
  - Gesamtaussage ist wahr, wenn beide Teilaussagen wahr sind
  - Symbol: $\wedge$
  - Beispiel: $a \wedge b$

- **Oder-Verknüpfung**
  - Gesamtaussage ist dann wahr, wenn mindestens eine der beiden Teilaussagen wahr ist
  - Symbol: $\vee$
  - Beispiel: $a \vee b$

Logische Verknüpfungen #3

- **Äquivalenz-Verknüpfung**
  - Gesamtaussage ist dann wahr, wenn beide Aussagen den gleichen Wahrheitswert haben
  - Symbol: $\equiv$ $\Leftrightarrow$
  - Beispiel: $a \equiv b$

- **Antivalenz**
  - Gesamtaussage ist wahr, wenn genau eine Teilaussage wahr ist
  - Symbol: $\neq$
  - Beispiel: $a \neq b$

Logische Verknüpfungen #3

- **Negiertes Exklusives Oder (XNOR)**
  - Gesamtaussage ist dann wahr, wenn beide Aussagen den gleichen Wahrheitswert haben
  - Symbol: $\equiv$ $\Leftrightarrow$
  - Beispiel: $a \equiv b$

- **Exklusives Oder (XOR)**
  - Gesamtaussage ist wahr, wenn genau eine Teilaussage wahr ist
  - Symbol: $\otimes$
  - Beispiel: $a \otimes b$

Logische Verknüpfungen #4

- **Implikation**
  - Gesamtaussage ist nur dann falsch, wenn erste Teilaussage wahr und zweite Teilaussage falsch ist
  - Entspricht in etwa der umgangssprachlichen Wenn-dann Formulierung (ist aber exakter, z.B. wenn die erste Teilaussage falsch ist)
  - Symbol: $\Rightarrow$
  - Beispiel: $a \Rightarrow b$

    a = „ich bestehe die Prüfung"

    b = „ich bin glücklich"

    Kann ich glücklich sein ohne die Prüfung zu bestehen? ✓

    Kann ich die Prüfung bestehen ohne glücklich zu sein? ↯

Wahrheitstabelen

- Tabellarische Darstellung der Funktion
  - Alle möglichen Eingangskombinationen
  - Resultierende Funktionswerte

		Operator				
a	b	Und	Oder	Äquivalenz	Antivalenz	Implikation
0	0	0	0	1	0	1
0	1	0	1	0	1	1
1	0	0	1	0	1	0
1	1	1	1	1	0	1

Regeln #1

- **Kommutativität**
  - $a \wedge b \equiv b \wedge a$
  - Und, Oder, Äquivalenz und Antivalenz sind kommutativ
  - Implikation ist nicht kommutativ

- **Assoziativität**
  - $(a \wedge b) \wedge c \equiv a \wedge (b \wedge c)$
  - Und, Oder, Äquivalenz und Antivalenz sind assoziativ
  - Implikation ist nicht assoziativ
  - Wichtig bei der Verknüpfung mehrerer Aussagen

Regeln #2

- **Tautologie**
  - Aussage, die immer wahr ist
  - Beispiele:
    - $a \vee \neg a$
    - $(a \Rightarrow b) \vee (b \Rightarrow a)$

- **Kontradiktion**
  - Aussage, die immer falsch ist
  - Beispiel: $a \wedge \neg a$

Boolesche Algebra #1

- Spezielle Algebra (Komplementärer, distributiver Verband)
- Zahlenmenge (Körper) mit Addition und Multiplikation
  - Zahlenmenge z.B. Wahrheitswerte
  - Addition ≡ Oder-Verknüpfung
  - Multiplikation ≡ Und-Verknüpfung

Boolesche Algebra #2

Definition:

Eine Menge B von Elementen, über der zwei Operationen $+$ und $*$ erklärt sind, ist genau dann eine Boolesche Algebra $(B; +, *)$, wenn für beliebige Elemente $a, b, c \in B=\{0,1\}$ folgende Axiome gelten:

(1) $a + b = b + a$
$a * b = b * a$

Kommutativität

(2) $0 + a = a$
$1 * a = a$

Nullelement 0 bzw. Einselelement 1 bzgl. + bzw. * existiert (neutrale Element)

(3) $(a + b) * c = (a * c) + (b * c)$
$(a * b) + c = (a + c) * (b + c)$

Distributivität einer Operation bezüglich einer anderen

(4) $a + \neg a = 1$
$a * \neg a = 0$

Zu jedem Element $a \in B$ existiert ein komplementäres Element $\neg a \in B$

Weitere Gesetze

- Assoziativitätsgesetz

- Idempotenz
  - $a + a = a$
  - $a * a = a$

- Absorption
  - $(a + b) * a = a$
  - $(a * b) + a = a$

- De Morgansche Regeln
  - $(\neg a * \neg b) = \neg (a + b)$
  - $(\neg a + \neg b) = \neg (a * b)$

De Morgansche Regeln

- Folgerung: Und und Oder lassen sich ineinander verwandeln
  - $a + b \;=\; \neg(\neg a * \neg b)$
    Variante (1) negieren
  - $a * b \;=\; \neg(\neg a + \neg b)$
    Variante (2) negieren

- Wird zur algebraischen Vereinfachung von Ausdrücken gebraucht

Eindeutigkeit

- Verschiedene algebraische Darstellungen der gleichen Funktion möglich:
  - $y = b * a + \neg a$
  - $Y = b + \neg a * \neg b$

- Beide Darstellungen sind nicht minimal:
  - $y = b + \neg a$

a	b	y
0	0	1
0	1	1
1	0	0
1	1	1

- Normalformen
- Verfahren zur Optimierung
  - Karnaugh-Veitch
  - Quine-McCluskey
  - Binäre Entscheidungsdiagramme

  → Thema im Teil „Entwerfen“

Boolesche Ausdrücke

- Die Elemente 0 und 1 sind Boolesche Ausdrücke
- Die Symbole/Variablen $x_1, \ldots, x_n$ sind Boolesche Ausdrücke
- Sind g und h Boolesche Ausdrücke, so auch
  die **Disjunktion** $(g+h)$,
  die **Konjunktion** $(g \cdot h)$
  und die **Negation** $(\sim g)$.
- Nichts sonst ist ein Boolescher Ausdruck.

Vereinbarung für das Schreiben

1. Negation ~ bindet stärker als Konjunktion ·
2. Konjunktion · bindet stärker als Disjunktion +

→ Klammern können häufig weggelassen werden,
ohne dass Mehrdeutigkeiten entstehen

Überblick der nächsten Wochen

=

- **Grundlagen**
  - Beschreibungen über „0“ und „1“ (Boolesche Algebra)
  - Beschreibungen von Schaltungen
- **Rechnen**
  - Digitale Schaltungen für Addition, Subtraktion, Multiplikation
- **Speichern & Steuern**
  - Sequentielle Schaltungen
  - Speicherelemente
  - Endliche Automaten
- **Prozessor**
  - Aufbau
  - Optimierungen
- **Entwerfen**
  - Synthese von allgemeinen Schaltungen
  - Logikminimierung